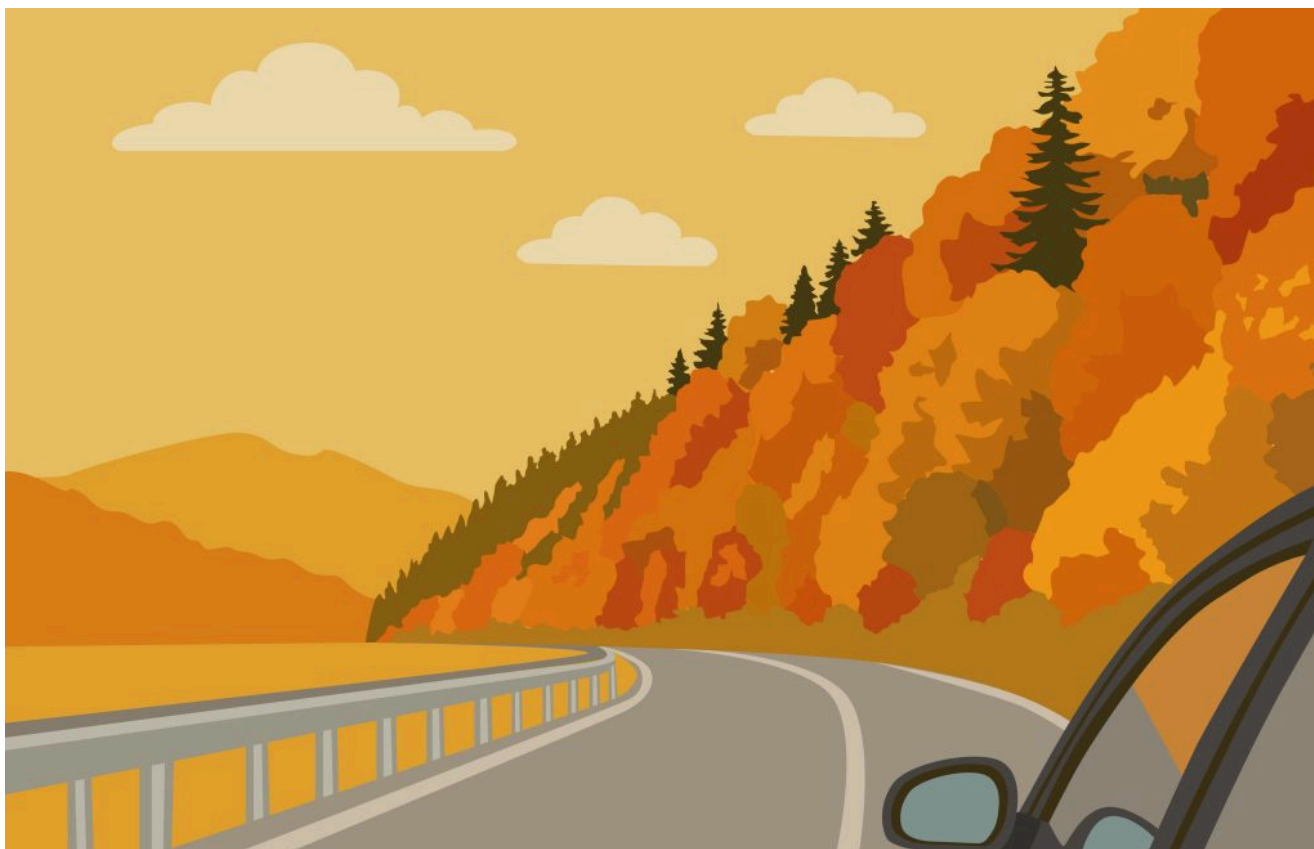


ЭКОСИСТЕМА CLOUD NATIVE / НАБЛЮДАЕМОСТЬ / ОТКРЫТЫЙ ИСХОДНЫЙ КОД

Нейтралитет к поставщикам — это не волшебство: пристальный взгляд на экосистему OpenTelemetry

Действительно ли OpenTelemetry не зависит от поставщика? Узнайте, в чем преимущества этого стандарта и с чего начинаются скрытые издержки при смене поставщика.

29 мая 2026 года, 10:00 [Адриана Виллела](#) и [Джош Ли](#)



Яна Кравчук для Unsplash+

[Dynatrace](#) спонсировала этот пост.

Экосистема **OpenTelemetry** (OTel) предоставляет нам не только стандартный формат данных и механизм передачи для генерации, обработки и экспорта телеметрических данных, но и гарантирует независимость от поставщика. Но что это значит на самом деле?

«Я до сих пор слышу, как люди говорят: «OpenTelemetry не зависит от поставщика, поэтому вы можете сменить поставщика в любой момент». ... Это все равно что сказать: «Поскольку существует формат vCard, легко перейти с iOS на Android».



Josh Lee · 1st

Open Source Advocate @ Altinity - ClickHouse®, Kubernetes, & ...
1mo · 🌐

I still hear people saying that "OpenTelemetry is vendor neutral so you can switch vendors any time you want to."

This is like saying that because the vCard format exists, it's easy to switch from iOS to Android. Sure, your core data will ultimately be portable, but you'll be missing a *lot* of other things.

Open formats are awesome — and essential for portability to be possible at all — but they are not a panacea that will save you from understanding your own use case.

[#opentelemetry](#) [#otel](#)

Это, конечно, заставило Адриану задуматься и привело к написанию **этого поста в блоге**, в котором она развивает мысли Джоша.



Dynatrace меняет подход к работе разработчиков, объединяя журналы, метрики, трассировки, телеметрию моделей искусственного интеллекта, данные об инфраструктуре и безопасности в единую масштабируемую платформу, которая интегрируется непосредственно в интегрированные среды разработки и конвейеры CI/CD.

Узнать больше →

ПОСЛЕДНИЕ НОВОСТИ ОТ DYNATRACE

24 июня 2026 года

Открытое строительство: как Dynatrace инвестирует в открытый исходный код, чтобы двигать отрасль вперед

23 June 2026

Как рабочие нагрузки на основе искусственного интеллекта меняют требования к логам, вынуждая применять новую стратегию

17 June 2026

УЗНАЙТЕ БОЛЬШЕ ОТ НАШЕГО СПОНСОРА

svo1975pvn1982@gmail.com

Отправить

Однако мы поняли, что нам есть что еще изучить по этой теме, а именно:

- Как опыт сообщества OpenTelemetry в области распределенной трассировки привел его к нейтральному отношению к поставщикам.
- Как структура стандарта OpenTelemetry делает его нейтральным по отношению к поставщикам.
- Где заканчивается нейтральность экосистемы OpenTelemetry по отношению к поставщикам.
- Заглядывая в будущее: можем ли мы сделать экосистему OpenTelemetry полностью нейтральной по отношению к поставщикам и нужно ли это делать?

Buckle up, kids! Let's start exploring!

TRENDING STORIES

1. LLMs create a new blind spot in observability
2. Observability overload is drowning engineers
3. After becoming cloud computing's telemetry standard, OpenTelemetry graduates into the AI infrastructure era

The OpenTelemetry community exists because of distributed tracing

It's helpful to understand why — why use the OpenTelemetry standard at all? Why use distributed tracing? Why care about vendor neutrality?

Let's start with openness. **Observability (and monitoring) begs to be open.** Specifically, when we undertake the task of monitoring all the different systems used by an organization, open protocols that can serve as translation layers become invaluable.

“Observability (and monitoring) begs to be open.”

But the OpenTelemetry ecosystem is somewhat new — and other attempts at universal monitoring formats have floundered. So why has the OpenTelemetry ecosystem seen such widespread adoption?

Distributed tracing is the key to understanding the OpenTelemetry ecosystem's success as a unifying force. Before microservices, monitoring meant logs and metrics; these signals had widely adopted and relatively open standards: e.g., **Prometheus** or **SNMP** for metrics, **syslog** or the ELK stack's JSON format for logs.

The open protocols in this case were straightforward: If each monitored entity could emit metrics and logs in some compatible format, they could be ingested and stored centrally by any number of tools.

As has been written about *ad infinitum* by Observability vendors, **microservices demand distributed tracing.** The information that could previously be gleaned from a single stack trace or log was now spread across dozens or hundreds of different processes.

Distributed tracing isn't just a change in scope — it's also a complete shift in context. To emit correct trace spans, each process needs to understand a small

expanded greatly.

This has enormous benefits. Reading a single distributed trace is *much* simpler than piecing together disparate logs based on tags. But it also comes at the cost of increased complexity that crosses the boundaries of individual technologies. All of a sudden, the **Java** people must care a lot more about the instrumentation of the Python people and vice versa.

The first solution to this increased complexity came from vendors with proprietary agents and protocols. Each vendor created instrumentation SDKs for as many programming languages, frameworks, and runtimes as possible — this duplicated effort. Before the OpenTelemetry standard came into existence, each vendor invested significant time, people, and money in creating their own instrumentation frameworks. It also created a lock-in effect for customers who had to embed proprietary SDKs in *every single software artifact* they shipped.

There's an infamous **xkcd cartoon about standards** that makes fun of how we try to fix having too many competing standards by creating a new standard. While that has held true in many cases, the OpenTelemetry standard has enabled format compaction. Instead of competing against each other, vendors were collaborating on a common standard. Soon, what differentiated vendors from one another was what they could do with the data ingested to provide value to their customers. We believe that this was possible because everybody working in the space agreed on some fundamental truths:

- Microservices demand more than traditional monitoring.
- Standardization is one of the core challenges of **monitoring and observability**.
- Open formats enable collaboration on standardization across technological and organizational boundaries.

In other words: *Observability begs to be open*.

What makes an OpenTelemetry tool vendor neutral?

Now that we've established the need for the vendor neutrality provided by the OpenTelemetry ecosystem, we are forced to ask: What does it *mean* for the ecosystem to be vendor-neutral? It means that *if you choose to send your observability data to a different vendor, you don't need to update your instrumentation*. THIS IS A BIG DEAL, because in the Before Times, changing

vendor than go through the pain of re-instrumenting code to work with a new vendor.

Thanks to the OpenTelemetry standard's vendor neutrality, you just need to update the destination you send your telemetry data to. In fact, if you're evaluating multiple vendors, you can even send the same data to different vendors at the same time (à la "vendor bake-off") to help you determine which vendor best suits your organization's needs.

"Thanks to the OpenTelemetry standard's vendor neutrality, you just need to update the destination you send your telemetry data to."

All this is made possible by the following components of the OpenTelemetry ecosystem:

- APIs & SDKs
- OTLP
- Collector

Let's dig into how each one of these provides an additional layer of flexibility and compatibility.

OpenTelemetry APIs and SDKs

The OpenTelemetry specification provides a standard definition of telemetry data and describes how to instrument code. The OpenTelemetry APIs and SDKs (more on those shortly) are defined in the specification. This ensures that users have a similar experience, regardless of the language they use.

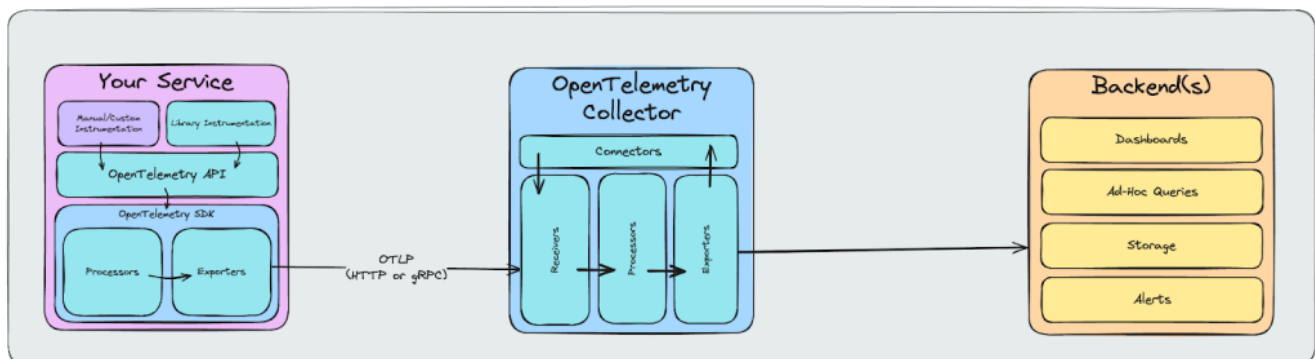
The OpenTelemetry application programming interface (API) defines the methods for instrumenting applications and serves as the entry point for instrumentation. Each language supported by OpenTelemetry has its own API implementation.

The OpenTelemetry software development kit (SDK) implements the API and determines *how* the telemetry is generated and correlated. Each language supported by OpenTelemetry has its own SDK.

whether it's the official OpenTelemetry SDK, a vendor SDK, or your own. (Yes: You can write your own OpenTelemetry SDK! 🤖) As long as they implement the OpenTelemetry API, it doesn't matter which one you choose.

This means that libraries (and your own code) can be agnostic about which SDK is used. This, in turn, empowers library vendors to provide out-of-the-box (OOB) instrumentation, which offers all the benefits of auto-instrumentation without the complexity or additional overhead. (Although auto-instrumentation is worth it if OOB-instrumentation is not provided!)

Not ready to commit to an SDK? No problem! Because the OpenTelemetry API also includes a minimal SDK. This allows you to instrument an application that will still run; however, keep in mind that no telemetry data will be sent to your backend until you plug in a proper SDK.



OTLP

OpenTelemetry Protocol (OTLP) is a vendor- and tool-agnostic specification for encoding, transmitting, and delivering OpenTelemetry data. Telemetry data emitted by the SDK uses OTLP, and most Observability backends now support OTLP and are strongly encouraged to ingest it. For those who do not, there are exporters available that convert OTLP data to a tool-specific format. OTLP supports both HTTP and gRPC.

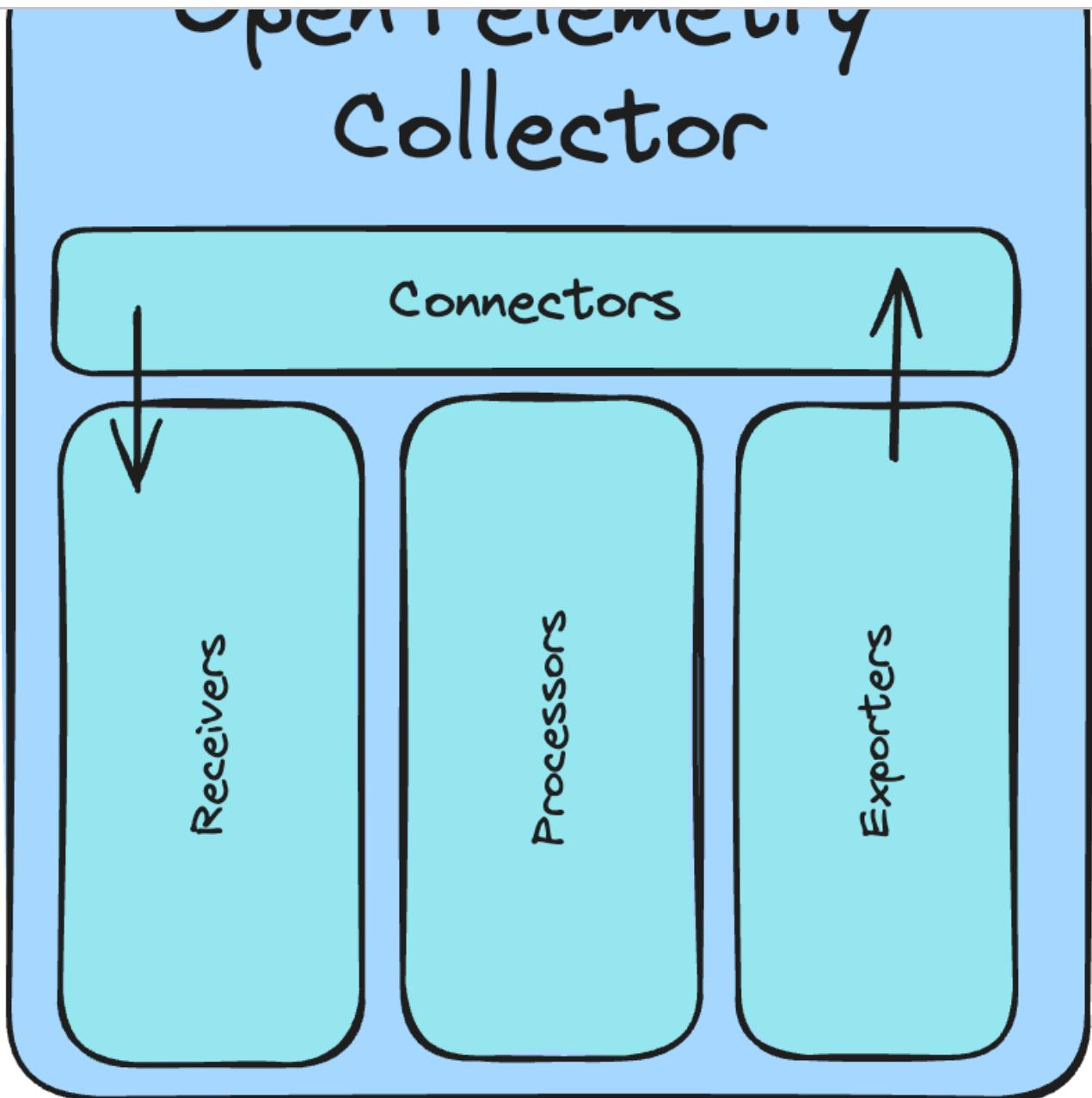
OTLP is a kind of magic sauce. There has been a burst of OTLP-compatible tools, which are all interoperable thanks to the shared protocol, format, and semantic conventions.

Collector

- **OpenTelemetry Collector** is a vendor-neutral binary that ingests, transforms, and exports data to one or more Observability solutions.

Advantages of the Collector:

- **Decouples the telemetry source from its destination.** The Collector acts as a proxy of sorts. Telemetry is sent directly to a single destination – the Collector – which then forwards it to one or more backends.
- **Acts as a single agent for multiple data types.** The Collector can ingest multiple data formats from multiple data sources, including applications and infrastructure.
- **Distributed telemetry **data to multiple backend systems**,** effectively allowing you to do a vendor bake-off (or let teams use a combination of tools).



The beauty of interoperability

Speaking of multicast — this is one of the most awesome uses of the OpenTelemetry specification, and it's not limited only to bake-off situations. The OpenTelemetry ecosystem's open formats mean that interoperable tools can be used together at *all times*.

Let's consider a few use cases.

Case 1: one tool for each signal

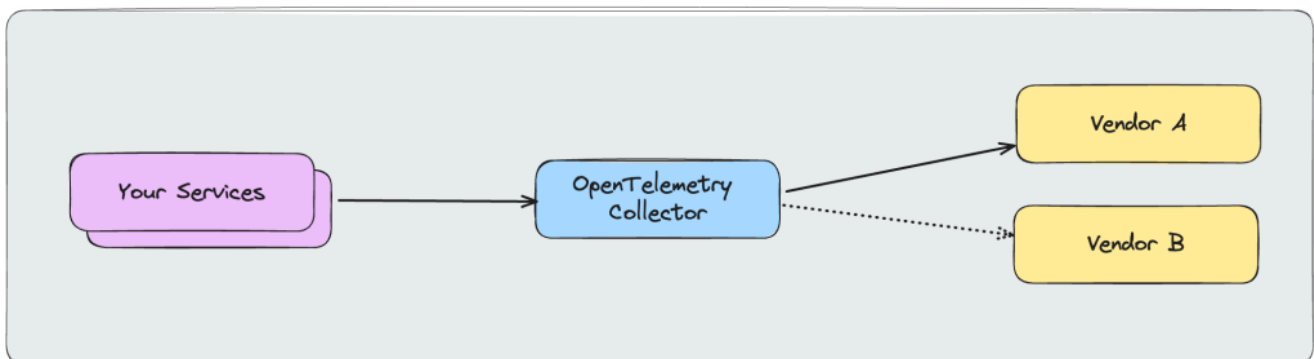
Because the Collector allows you to send telemetry to one or more backends at the same time, you can choose to send each signal to a different destination.



Use Case 2: one (or more) tools for all signals

Perhaps you have an all-in-one backend that supports the OpenTelemetry signals. In that case, you can send all signals to the single backend.

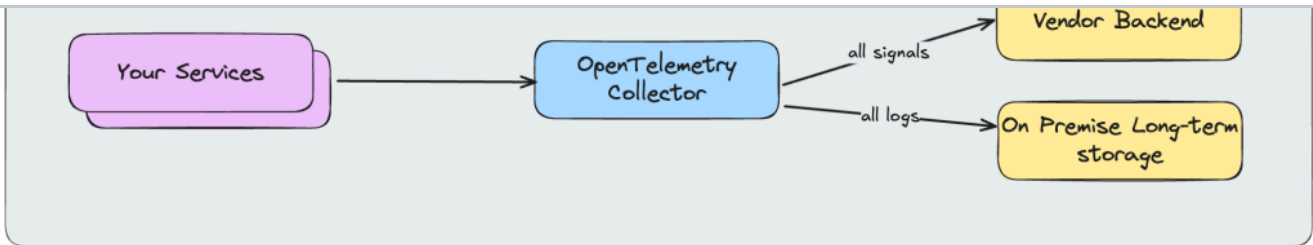
If you're between backends and don't know which one to choose, you can send your telemetry data to two all-in-one backends at the same time, and compare them, bake-off style.



Use Case 3: one tool for everything else, and a specific tool for something else

Many organizations require long-term data storage for compliance reasons. In that case, you might choose not only to send your telemetry data to your favorite backend but also to send it to an on-premises tool for long-term storage.

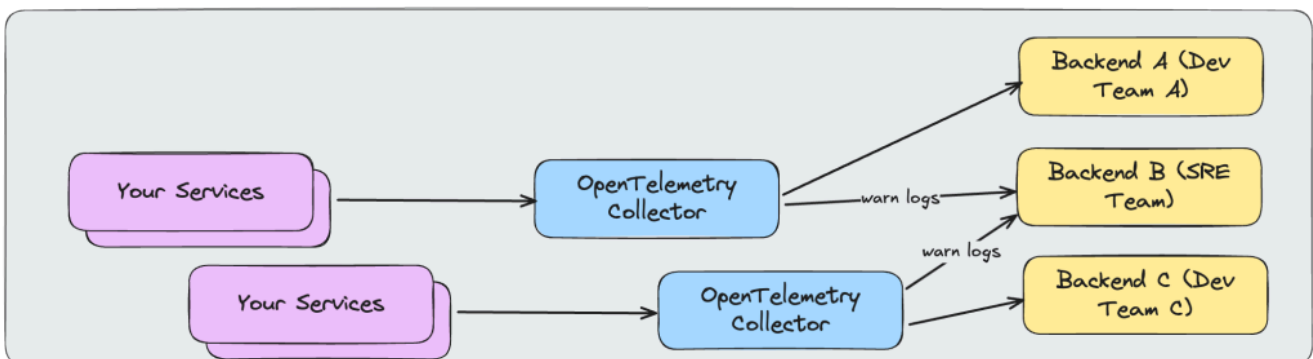
Or suppose that you have an observability vendor that you absolutely love, and they meet 95% of your requirements. Still, they are missing one critical tool (e.g., not all commercial solutions support Profiling). The OpenTelemetry ecosystem helps you fill those gaps with other compatible open source or vendor tools.



Use Case 4: specific pipelines for specific teams

Suppose that your company uses backend A for production. Development teams might also want to use OpenTelemetry signals to troubleshoot their own code. These teams might therefore use a local open-source observability tool, such as [otel-tui](#), [OTel Front](#), or [OTel Viewer](#), to help with troubleshooting.

Or consider another scenario. Suppose you work at Company A. It merges with Company B. Company A uses backend X, and Company B uses backend Y. Until both organizations are fully integrated, you might find the merged organization using two different backends at the same time.



Using the OpenTelemetry standard means you can have everything you want. You can have your cake and eat it too... (As long as you are willing to pay for storing two cakes.)

The vendor switch: know what you're getting into

Now, what about those non-vendor-neutral parts of the OpenTelemetry ecosystem? Is that even a thing?

When it comes to data ingestion, the OpenTelemetry protocol is absolutely vendor-neutral.

and courses, and all she had to do was repoint her OpenTelemetry collector to a different backend. Boom. Magic. ✨ She was able to see her traces, logs, and metrics in the Dynatrace UI, just like she had in the Lightstep UI.

“What nobody talks about, however, is that it’s not just about data ingestion, is it?”

What nobody talks about, however, is that it’s not *just* about data ingestion, is it? We must also consider things like:

- Look & feel
- Storage & historical data
- Queries & dashboards
- Alerts & SLOs
- Infrastructure-as-Code (IaC)

Look & feel

If you’ve been working with Vendor X for a while, you’ve been used to doing things in a certain way. A certain workflow, if you will. Whether it’s your personal workflow or one your SRE team follows. Bottom line: using a new vendor means getting used to a new UI and the vendor’s specific nuances. Maybe Vendor X has features that Vendor Y doesn’t, which let you do some cool stuff with your OpenTelemetry data — which means that you must familiarize yourself with these new features to get the most out of them.

Storage & historical data

The observability business model is data-driven. — practically every vendor charges based on some combination of *ingested* and *retained data*. As far as we know, no vendor allows historical telemetry data to be exported, which means that if you switch vendors, you lose your historical data unless you also send your telemetry data to an internally managed datastore at the same time*.

(*) *But you can do this now, thanks to the OpenTelemetry protocol— cool*

Queries & dashboards

that when you switch vendors, you can't simply lift and shift your dashboards. You must learn a new dashboarding interface, a new query language, and then translate your old vendor's dashboard into the new vendor's dashboard. Bottom line: they require time and effort to build out.

Like all things, there are exceptions to the rule:

- Grafana Dashboards can be reused with the same data source type. For example, **there are multiple PromQL-compatible data sources**
- **Perses** offers an embeddable **license** and an open format for dashboards that supports PromQL

Alerts & SLOs

Some Observability vendors provide the ability to alert on service-level objectives (SLOs), which are themselves informed by OpenTelemetry metrics. If you choose to switch vendors, you must re-create your alerts and your SLOs. That is, if that vendor even supports alerts and SLOs.

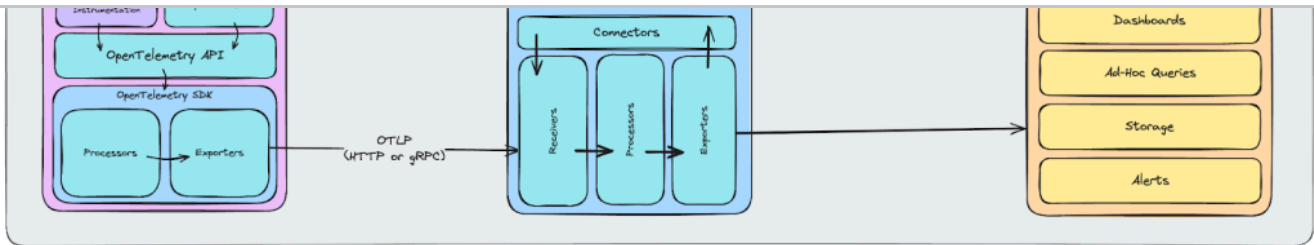
IaC

Many **major Observability vendors have their own Terraform providers**, which are super helpful for automating configuration across various aspects of your Observability vendor tool. Unfortunately, changing Observability vendors means changing Terraform providers as well. Also, not all Observability vendors' Terraform providers let you configure the same things, which means this is, again, not a "lift and shift" situation.

It should be noted that, thanks to agentic AI, some tasks, such as creating queries and dashboards and writing IaC code, greatly reduce migration efforts when moving from one vendor to another.

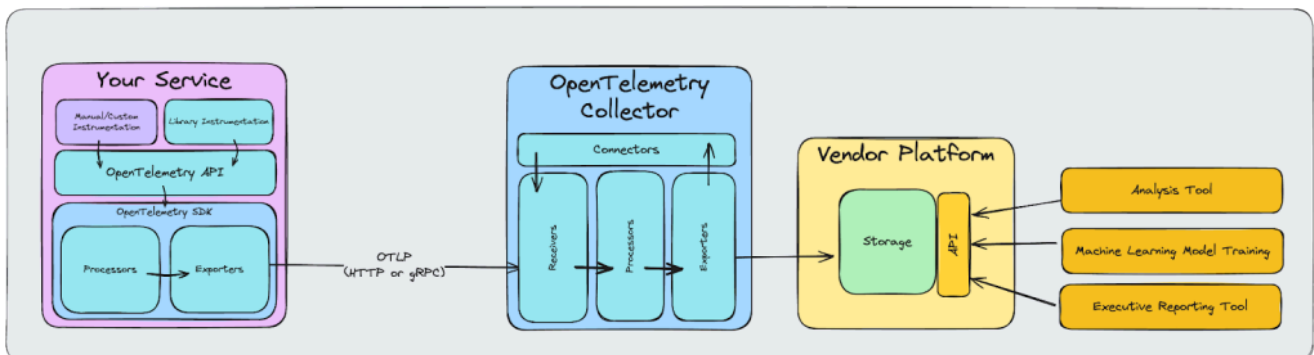
A future vision for vendor-neutrality

Every Observability Platform has its areas to shine; if they weren't unique, there'd be no reason to switch. But they also share some fundamental characteristics. We need somewhere to put the telemetry we've collected (**Storage**), a way to explore that data (**Ad-Hoc Queries**), a way to share those explorations with our teams and future selves (**Dashboards**, usually), and a way to be notified when things aren't **d (Alerts)**.



While the OpenTelemetry project itself does not include any backend components, there are several open-source options, especially for storage, querying, and visualization.

But as we've also discussed, interoperability isn't just about switching. What if our favorite vendor made their data store accessible via a universally available API, allowing us to store one copy of our observability data as a Single Source of Truth, while making it accessible to multiple analysis tools *at the same time*? This would allow us to add specialized analysis tools, in-house model training, customized reporting, and more, all from a single API – we think that would be neat.



Can we live up to the promise of vendor neutrality?

The OpenTelemetry community has come a long way since it first came onto the scene. It emerged as a vendor-neutral solution to the challenge of distributed tracing, and has grown into the *de facto* standard for generating, collecting, and exporting Observability data.

We credit the strength of the OpenTelemetry project to a few key factors:

- The fact that observability is open. End-users have demanded it.
- Support – since its inception – of multiple vendors. No vendor takes up the spotlight.

And it continues to evolve. Just a few years ago, OpenTelemetry tooling was predominantly used in the instrumentation layer, with vendors providing proprietary exporters to speak with their agents. Now, OTLP is widely supported, and running OpenTelemetry Collectors instead of or alongside vendor agents is commonplace.

The open source ecosystem has also embraced the OpenTelemetry community. Jaeger was originally built around its own (open) tracing protocol. **It now** natively supports the OpenTelemetry protocol. And Prometheus, a popular tool, **is now co-evolving with the OpenTelemetry** ecosystem so that the benefits of shared standards can be enjoyed everywhere.

Will we continue to see more such changes? Only time will tell, but the future is looking bright.

Until then, remember this: the wonderful thing about the OpenTelemetry ecosystem is choice. Vendor neutrality and **open standards enable** so much more than “switching vendors” one day — although they certainly help with that too. The benefits of open formats are portable data and interoperable tools, allowing you to compose the observability solution you need *today*.

The key is — does a solution enable you to ask meaningful questions, get useful answers, and act on what you’ve learned? Because at the end of the day, if your applications and systems are more reliable, that’s what matters most.

TNS



Adriana Villela is a blogger, host of the Geeking Out podcast, CNCF Ambassador, OpenTelemetry Community Manager, and maintainer of the OpenTelemetry End User SIG. By day, she focuses on Observability and OpenTelemetry, as a Principal Developer Advocate at Dynatrace. By...

[Read more from Adriana Villela →](#)



leading software projects for clients across various industries. Josh is also a...

[Read more from Josh Lee →](#)

SHARE THIS STORY



TRENDING STORIES

1. LLMs create a new blind spot in observability
2. Observability overload is drowning engineers
3. After becoming cloud computing's telemetry standard, OpenTelemetry graduates into the AI infrastructure era
4. How AI Can Help IT Teams Find the Signals in Alert Noise
5. Vendor neutrality isn't magic: A hard look at the OpenTelemetry ecosystem

INSIGHTS FROM OUR SPONSOR



Dynatrace redefines developer experience by unifying logs, metrics, traces, AI model telemetry, infrastructure, and security data into a single, scalable platform that integrates directly into IDEs and CI/CD pipelines.

[Learn More →](#)

Log management for AI workloads: How to bring your logs and telemetry plan into the AI-first century

June 2026

How AI workloads are changing what logs must deliver, forcing a new strategy

17 June 2026

Dynatrace Release Radar 05.26

16 June 2026

Moving from insight to action: How Dynatrace and AWS are reshaping cloud operations

16 June 2026

What's new in Dynatrace SaaS version 1.341

16 June 2026

TNS DAILY NEWSLETTER

Receive a free roundup of the
most recent TNS articles in
your inbox each day.

svo1975pvn1982@gmail.com

SUBSCRIBE

The New Stack does not sell your information or share it with unaffiliated third parties. By continuing, you agree to our [Terms of Use](#) and [Privacy Policy](#).

ARCHITECTURE

Cloud Native Ecosystem
Containers
Databases
Edge Computing
Infrastructure as Code
Linux
Microservices
Open Source
Networking
Storage

OPERATIONS

AI Operations
CI/CD
Cloud Services
DevOps
Kubernetes
Observability
Operations
Platform Engineering

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

FOLLOW TNS



ENGINEERING

AI
AI Engineering
API Management
Backend development
Data
Frontend Development
Large Language Models
Security
Software Development
WebAssembly

CHANNELS

Podcasts
Ebooks
Events
Webinars
Newsletter
TNS RSS Feeds



Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

[Disclosures](#)

[Terms of Use](#)

[Advertising Terms & Conditions](#)

[Privacy Policy](#)

[Cookie Policy](#)